

NETWORK PROTOCOLS AND WIRESHARK

1. Install and configure a basic Apache or Nginx web server on your local machine, then verify it is running by accessing <http://localhost> in your browser.

->Option A — Apache on Ubuntu/Linux

Install Apache:

```
sudo apt update
```

```
sudo apt install apache2
```

Start it:

```
sudo systemctl start apache2
```

```
sudo systemctl enable apache2
```

Check its status:

```
sudo systemctl status apache2
```

Open your browser and visit:

<http://localhost>

You should see the **Apache2 Ubuntu Default Page**.

To create your own simple homepage:

```
sudo nano /var/www/html/index.html
```

Enter:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>My Web Server</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Welcome to My Web Server</h1>
```

```
  <p>Apache is working successfully!</p>
```

</body>

</html>

Save the file and refresh <http://localhost>.

Windows alternative

You can install Apache through a Windows Apache distribution such as XAMPP, then start Apache from the XAMPP Control Panel and open.

2. Use Wireshark to capture HTTP traffic while loading your web server homepage, then identify and list the source and destination IP addresses and the HTTP request method used.

->Start **Wireshark** and select the network interface that is carrying your traffic.

Start the capture.

Then open:

<http://localhost>

Refresh the page several times.

Useful Wireshark filter

For HTTP traffic:

http

If you are specifically looking for GET requests:

http.request.method == "GET"

You should find a packet similar to:

GET / HTTP/1.1

Host: localhost

What to record

Select the HTTP packet and examine the IP header.

For a local web server, you may see something like:

Field	Example
Source IP	127.0.0.1
Destination IP	127.0.0.1

Field	Example
Protocol	TCP/HTTP
Destination Port	80
HTTP Method	GET

Important: Your actual capture may show 127.0.0.1 because both the browser and web server are running on the same computer.

If you access the server using its LAN IP instead, the source/destination addresses will be different.

- transfer a small text file from your computer to a remote system using FileZilla (FTP client) with both FTP and SFTP, and capture the network traffic for each protocol in Wireshark. Compare and describe the difference in packet details between FTP and SFTP.

Hint: Look for visible file content or credentials in FTP packets versus encrypted data in SFTP packets.

-> For this experiment, you need an FTP/SFTP server that you are authorized to access. A convenient approach is to run one on a second VM or use a test server under your control.

FTP

Configure FileZilla with:

Protocol: FTP

Host: <your FTP server IP>

Port: 21

Encryption: Plain FTP (for lab testing)

Username: <test username>

Password: <test password>

Transfer a small file such as:

test.txt

Start Wireshark before transferring it.

Use:

ftp

or:

```
tcp.port == 21
```

What you may observe

With **plain FTP**, protocol information is transmitted without encryption.

Depending on the connection and FTP operation, Wireshark can potentially display things such as:

USER <username>

PASS <password>

and FTP commands such as:

STOR test.txt

RETR test.txt

LIST

The contents of an ordinary text file may also be visible in the captured TCP stream.

Do not use real passwords or confidential files for this experiment.

SFTP

SFTP is different from FTP. It operates through **SSH**, normally using TCP port **22**.

Configure FileZilla:

Protocol: SFTP - SSH File Transfer Protocol

Host: <your SFTP server IP>

Port: 22

Username: <test username>

Authentication: Password/key

Transfer the same test.txt file.

Wireshark filter:

ssh

or:

```
tcp.port == 22
```

You will generally see:

TCP

SSH

Encrypted packets

but not readable FTP commands, usernames/passwords, or the text contents of the transferred file.

Comparison

Feature	FTP	SFTP
Normal port	21	22
Underlying protocol	FTP	SSH
Encryption	No encryption with plain FTP	Encrypted
Username/password visibility	Potentially visible	Encrypted
File contents	Potentially readable	Encrypted
Wireshark payload	FTP commands/data can be visible	Mostly encrypted SSH payload

4. Analyze a public website (such as <https://www.flipkart.com>) in Wireshark and find a packet that uses HTTPS. Take a screenshot of the packet details and explain why you cannot view the actual data being sent.

-> Open an HTTPS website you are authorized to inspect, such as:

<https://www.flipkart.com>

Start Wireshark before opening the website.

You can filter HTTPS/TLS traffic using:

tls

or:

tcp.port == 443

You should see packets containing information such as:

Internet Protocol

Source: <your IP>

Destination: <website/server IP>

Transmission Control Protocol

Src Port: <ephemeral port>

Dst Port: 443

Transport Layer Security

TLS Application Data

What to screenshot

Select a TLS packet and expand:

Internet Protocol

Transmission Control Protocol

Transport Layer Security

Your screenshot should show the packet details.

Why can't you see the actual webpage data?

HTTPS uses **TLS encryption**.

The network capture can still reveal metadata such as:

- Source/destination IP addresses
- TCP ports
- Packet sizes
- Timing
- TLS protocol information

But the actual HTTP contents are encrypted.

For example, instead of seeing:

GET /products HTTP/1.1

Host: example.com

the packet payload will generally appear as encrypted TLS application data.

This is one of the major security advantages of HTTPS.

5. Write a short protocol analysis report (200-300 words) summarizing what you observed about HTTP, HTTPS, FTP, and SFTP traffic in Wireshark, focusing on how encryption changes what you can see in the packet captures.

-> **Protocol Analysis Report**

During the Wireshark experiment, I analyzed HTTP, HTTPS, FTP, and SFTP traffic to understand how encryption affects network communication. When accessing the local Apache web server through HTTP, Wireshark allowed me to observe TCP packets and the HTTP request. The HTTP GET method was visible, along with information such as source and destination IP addresses and port 80. This demonstrated that HTTP transmits application-layer information without encryption.

I then compared FTP and SFTP while transferring a test text file using FileZilla. With plain FTP, Wireshark could identify FTP commands and protocol information. Depending on the configuration, sensitive information such as login credentials and transferred text data may also be visible in the capture. This demonstrates why unencrypted FTP is unsuitable for transmitting confidential information over untrusted networks.

SFTP produced a significantly different capture. SFTP operates through SSH, normally using TCP port 22. Wireshark could identify the SSH connection and encrypted packets, but the actual commands, credentials, and file contents were not readable from the normal packet capture.

Finally, I examined HTTPS traffic. HTTPS uses TLS encryption, normally over TCP port 443. Wireshark could still show network metadata, including IP addresses, ports, packet sizes, and TLS information, but the actual HTTP application data was encrypted.

Overall, the experiment demonstrated that **HTTP and plain FTP expose substantially more application-layer information**, whereas **HTTPS and SFTP encrypt the transmitted data**, making captured payloads unreadable without appropriate decryption keys and session information.